

DT_두산에너지빌리티: 보행 로봇 제어 시스템 개발

요약 정리본 v3 · 기준: 과제 요구사항 원문 + PLAN.md (2026-08-31, Phase 0 실측 후 Jazzy/Harmonic으로 스택 변경)
실행 전제: macOS + Docker(Ubuntu 24.04 arm64 네이티브) / ROS 2 Jazzy / Gazebo Harmonic / Unitree Go2
수행 시간: 필수 205h + 보너스 60h

v2 대비 변경점

① Humble+Gazebo Classic → **Jazzy + Harmonic** ② CHAMP 조건부화 및 대체 경로 명시 ③ **열화상 커스텀 플러그인 자작 취소** — Harmonic 네이티브 thermal 센서 사용 ④ **ros_gz_bridge** 학습 항목 신설

③④의 인원 배분은 역할 4트랙(환경 / 제어 / 비전 / 시스템) 기준. 1인 수행이면 “팀원 배분”이 아니라 **학습 순서**로 읽으면 된다.

① 과제 분석 — 우리가 해결해야 할 문제는 무엇인가?

1-1. 핵심 문제

발전소 현장은 고온 배관 구역, 유독 가스 누출 가능 구역, 좁은 철제 계단과 미끄러운 바닥처럼 **사람이 들어가기 위험한 구역**이 상당수다. 이런 구역일수록 점검 주기가 짧아야 하고, 설비 상태를 알려주는 계기 상당수는 **디지털 통신이 불가능한 아날로그 압력계**다. 결과적으로 현재 점검은 사람의 눈과 손, 종이 기록에 의존하며 ① 작업자 안전 리스크, ② 점검 데이터의 비정형·비실시간성, ③ 이상 징후 조기 발견 실패로 이어진다.

본 과제는 이 문제를 “**사람 대신 4족 보행 로봇을 보내고, 결과를 Digital Twin 관제 화면에서 실시간으로 확인한다**”로 해결한다. 실제 로봇 하드웨어를 쓸 수 없으므로 가상 발전소(Virtual Plant)에서 전 과정을 구현·검증한다.

1-2. 사용자 (Stakeholder)

구분	대상	기대하는 것
1차	발전소 정비팀	위험 구역 무인 점검, 계기 수치 자동 기록
2차	로봇솔루션팀	실기 투입 전 보행·인식 알고리즘 검증 환경
3차	DX추진팀	설비 상태 데이터의 디지털화 및 관제 통합

1-3. 최종 결과물

module1_locomotion / module2_inspection / module3_gas_safety / simulation / dashboard / docs를 포함한 **GitHub Repository 1식**. 모델링 최소 조건(복도 45×3.5×4m, 공장 25×18×7m, 혐지 방해물 복도 3·공장 5, 로봇개 1, 공장기계 1)을 충족하는 3D 환경과 REPORT.md / README.md 포함.

1-4. Digital Twin 4요소 ↔ 우리 구현물 매핑

요구사항 #9의 첫 Digital Twin 질문(“각 구성 요소는 어떤 역할을 수행하며...”)에 대한 직접 답변.

DT 구성 요소	요구사항이 정의한 역할	우리 구현 (Harmonic 기준)	실제 발전소 반영 요소
Virtual Plant	모든 데이터가 적용되는 가상 환경	simulation/worlds/ — 복도·공장 SDF 월드, 계단, 파이프 잔해, 공장기계, 가스 탱크. 발열 설비에는 heat signature 온도 태그	공간 치수, 통행 폭, 계단 경사, 장애물 밀도, 조명, 배관 표면 온도
Robot Twin	로봇 상태 및 센서 데이터 생성	Unitree Go2 URDF + gz_ros2_control + gpu_lidar / rgbd_camera / thermal 카메라 / IMU / 가스 센서(자체 노드)	관절 구성, 센서 장착 위치·시야각, 배터리 소모 모델
Inspection Data	로봇이 수집하는 설비 상태 데이터	/inspection/gauge_reading, /inspection/thermal_alert, /gas/concentration	아날로그 계기 수치, 배관 표면 온도(K), 가스 농도 분포
Monitoring System	분석 결과 및 로봇 상태 시각화	dashboard/ — rosbridge + 웹 대시보드 (보너스①과 통합)	로봇 위치, 임무 상태, 점검 이력, 가스 농도맵

1-5. 환경 제약과 대응 — 발표 핵심 소재

개발 머신이 **macOS(Apple Silicon)**라 ROS 2 + 시뮬레이터를 네이티브로 실행할 수 없다. 이 제약을 우회하는 대신 부가 목표로 전환했다.

“과제 요건 충족”에 더해 **누구나 docker compose up 한 번으로 재현 가능한 무료 환경**을 목표에 추가한다.

이 결정을 계획 단계에서 확정하지 않고, Phase 0에서 실측으로 검증한 뒤 스택을 변경했다. 이 의사결정 기록 자체가 ①의 가장 강력한 발표 재료다.

Phase 0 검증 결과와 스택 변경 (2026-08-31)

	당초 계획	실측 결과	최종 확정
ROS 배포 판	Humble	osrf/ros:humble-desktop이 amd64 전용 → Apple Silicon에서 에뮬레이션, 빈 월드 RTF 0.5	Jazzy
시뮬레이터	Gazebo Classic	Humble arm64 저장소에 Gazebo Classic도 ros-gz-sim도 빌드 없음	Gazebo Harmonic (gz-sim8)
근거	—	Jazzy + Harmonic은 arm64에서 전체 스택(ros-gz-sim, gz_ros2_control, Nav2, Foxglove, rosbridge) 제공	arm64 네이티브 실행

파급 효과 (양방향)

- **불리:** CHAMP 등 Humble 기준 4족 오픈소스 스택은 Jazzy 포팅 검증이 필요 → \$5-4 리스크 1순위.
- **유리:** Gazebo Harmonic에는 열화상 카메라가 네이티브로 존재(type="thermal" + gz-sim-thermal-sensor-system). Classic 기준의 “열화상 합성 플러그인 자작”이 불필요해졌다.

이 두 파급은 문제 분해와 시간 배분을 바꾸므로 ②·⑤에 반영했다.

1-6. 프로젝트 한 문장 정의

우리는 macOS + Docker(arm64 네이티브) 위에서 ROS 2 Jazzy와 Gazebo Harmonic으로 발전소 Digital Twin을 구축하고, 4족 보행 로봇이 험지를 스스로 통과하고(Locomotion), 아날로그 계기와 과열 배관을 눈으로 읽어내며(Inspection AI), 가스 누출원을 추적한 뒤 안전하게 복귀하는(Gas/Safety) 자율 점검 시스템을 개발하여, 그 모든 상태와 결과를 웹 관제 화면에 실시간 시각화한다.
--

② 문제 분해 — 이 문제를 해결하려면 무엇을 해야 하는가?

2.1 최상위 문제 분해

가상 플랜트 구축 → 이동 능력 확보 → 현장 데이터 해석 → 임무 자율 판단 → Digital Twin 관제 통합

단계	대분류	해결해야 할 질문
P1	Virtual Plant 구축	발전소를 어떻게 가상 공간에 재현하고 센서 데이터를 만들 것인가
P2	Locomotion (이동 능력)	험지에서 넘어지지 않고 목표 지점까지 갈 수 있는가
P3	Inspection AI (데이터 해석)	로봇의 눈으로 설비 상태를 정량 수치로 바꿀 수 있는가
P4	Gas/Safety (임무 자율)	스스로 탐색하고 위험 시 스스로 복귀할 수 있는가
P5	Digital Twin 통합	모든 결과를 하나의 관제 화면에서 실시간으로 볼 수 있는가

2.2 세부 문제 분해

P1. Virtual Plant 구축

- P1-1 발전소 3D 환경 모델링 (복도 45m × 3.5m × 4m, 공장 25m × 18m × 7m)
- P1-2 4족 보행 로봇 모델링 (URDF → USD 변환, 관절 drive 설정)
- P1-3 센서 모델 구성 (LiDAR, Depth, IMU, RGB, 열화상 대체 모델, 가스 센서)
- P1-4 ROS 2 인터페이스 정의 (토픽·메시지 타입·QoS 계약 확정)

P2. Locomotion Control

- P2-1 지형 인지: 포인트클라우드 → Elevation Map 생성 및 노이즈 제거
- P2-2 발 디딤 위치 결정: 평면 분할 → 안전 후보 추출 → 지형 비용 기반 재배치
- P2-3 MPC 보행 제어기: SRBD 모델링 → QP 정식화 → 예측 구간·가중치 튜닝
- P2-4 보행 모드 전환: Trot / Crawl 전환 기준 및 전이 로직
- P2-5 Fall Recovery: 넘어짐 판정 → 기립 시퀀스

P3. Inspection AI

- P3-1 게이지 검출 및 ROI 정규화
- P3-2 바늘 각도 산출 → 수치 변환 (목표 오차 5% 이내)
- P3-3 열화상·RGB 정합 (캘리브레이션 → 시차 보정)
- P3-4 과열 판정 및 시각화 오버레이

P4. Gas/Safety

- P4-1 가스 확산·기류 시뮬레이션 환경 구축
- P4-2 순찰 경로 생성: Viewpoint 비용 설계 → TSP 순서 최적화
- P4-3 누출원 추적: 농도 구배 추정 → 이동 방향 결정 → 국소 최적 탈출
- P4-4 자율 복귀: 배터리·통신 트리거 → 장애물 회피 경로 계획

P5. Digital Twin 통합

- P5-1 모듈 간 통신 구조 설계 (Topic / Service / Action 배분, QoS)
- P5-2 시간 동기화 및 데이터 정합
- P5-3 관제 화면 구성 (로봇 상태, 점검 결과, 가스 농도 맵)
- P5-4 통합 성능 평가 (넘어짐 횟수, 인식 오차율, 지연 시간)

2.3 선후관계

P1 Virtual Plant → P2 Locomotion
P1 → P3 Inspection AI
P1 → P4 Gas/Safety
P2 → P4 (탐색은 이동 전제)
P2 · P3 · P4 → P5 Digital Twin 통합

- P1은 절대 선행: 시뮬레이션 환경 없이는 나머지 모듈 검증이 불가능하다.
- P2 · P3 · P4는 원칙적으로 병렬. 단 P4는 P2에 실질 의존.
- P5는 최종 통합이나, 인터페이스 정의(P1-4)는 착수 시점에 확정한다.

2-1. 전체 흐름 (DT 데이터 흐름을 작업 단위로 치환)

개발환경 구축 → Virtual Plant 구축 → Robot Twin(센서·보행) → 인지(Elevation/영상/농도)
→ 임무 자율화(순찰·추적·복귀) → 데이터 통신 규격 → 관제 시각화 → 통합·검증·문서화

2-2. 의존 관계 그래프

[P0] Docker arm64 / ROS 2 Jazzy / Gazebo Harmonic / Foxglove
| ← 없으면 아무것도 실행 불가
▼
[P1] Virtual Plant + Robot Twin 스폰
(월드 SDF, 온도 태깅, Go2 URDF, 센서 + ros_gz_bridge)
| ← 지형 없으면 Elevation Map 불가, 브리지 없으면 ROS 노드가 데이터를 못 봄
▼
[P2] Module 1 · Locomotion — 나머지 모듈의 "이동 인프라"
| Terrain Mapping → stage1 보행 확보 → stage2/3 MPC → Fall Recovery
| ← 여기서부터 병렬
▼
[P3] Inspection [P4] Gas/Safety
|
▼
[P5] 통신 규격 확정 + Monitoring System + 통합 시나리오
▼
[P6] 보너스 ②③④ → [P7] 검증·문서화

2-3. 선후관계의 근거 (발표 시 강조)

- **P2가 병목이자 인프라:** 로봇이 안정적으로 걷지 못하면 점검도 가스 추적도 시작할 수 없다. Module 1은 독립 모듈이 아니라 Module 2·3의 전제조건이라 55h로 가장 크게 배분했다.
- **P1에 ros_gz_bridge가 들어가는 이유:** Harmonic은 gz-transport를 쓰므로 센서 데이터가 그대로 ROS 2 토픽으로 오지 않는다. 브리지 설정이 Robot Twin 완성의 일부이며, 이것 P2로 미루면 Elevation Map 개발이 통째로 막힌다.
- **P3/P4는 병렬 가능:** 가스 확산은 시뮬레이터와 무관한 자체 ROS 2 노드(가우시안 플룸)로 구현한다.
- **P5를 따로 댄 이유:** 세 모듈 동시 구동 시의 병목·시간 동기화는 모듈을 다 만든 뒤 대응하면 늦다. 통신 규격은 P3 착수 시점에 초안을 확정하고 P5에서 검증만 한다.

2-4. Module 1 내부의 3단 리스크 분해

MPC는 실패 확률이 가장 높은 항목이라 **완주 보장선과 도전선을 분리**했다.

단계	내용	성격	실패 시 영향
stage1 champ_tuning	기성 4족 보행 스택 + 지형 적응 파라미터 + Trot/Crawl 게이트 전환	완주 보장선 — 이 시점에 과제 필수 요건 충족	프로젝트 중단
stage2 simple_mpc_py	SRBD 모델 기반 간이 MPC를 Python 직접 구현, Prediction Horizon 실험	이해도 확보	없음
stage3 convex_mpc_cpp	Convex MPC(MIT Cheetah 방식) C++ 구현	도전 · 타임박스 10h	없음, 보고서에 시도·한계 기술

Jazzy 전환으로 생긴 stage1의 새 리스크와 이중화

CHAMP는 Humble 기준 스택이라 Jazzy에서 동작한다는 보장이 없다. 완주 보장선이 흔들리면 프로젝트 전체가 흔들리므로 stage1을 이중화했다.

경로	내용	판정 시점
1-A (우선)	CHAMP 커뮤니티 Jazzy 포크 검증 후 채택	P0 말 ~ P1 초에 판정 (P2 착수 전)
1-B (대체)	gz_ros2_control + 자체 게이트 생성기(정적 보행 궤적 + IK)	1-A 실패 시 즉시 전환

1-B가 손실이 아닌 이유: 자체 게이트 생성기의 발끝 궤적·역기구학 코드는 stage2의 간이 MPC와 그대로 공유된다. 1-B로 가면 stage1과 stage2가 한 코드베이스로 합쳐져 총 작업량이 줄고, “왜 직접 만들었는가”라는 심층 인터뷰 답변도 강해진다. 대신 stage1 튜닝 15h 중 일부가 구현으로 옮겨가므로 **stage3(도전) 포기 가능성**을 미리 열어 둔다.

2-5. Gazebo Harmonic 경로에서 추가·삭제된 작업

작업	v2 (Classic 가정)	v3 (Harmonic 확정)	시간
열화상 카메라	커스텀 플러그인 자작	취소 — type="thermal" + gz-sim-thermal-sensor-system	8h → 2h
발열 오브젝트 정의	(플러그인에 포함)	신설 — 배관·기계에 heat signature / min_temp·max_temp	+2h (P1)
저조도·모션블러 주입	필요	유지 — 조명 엔티티 제어 + 이미지 후처리 노드	3h
ros_gz_bridge 구성	Classic은 불필요	신설 — gz-transport ↔ ROS 2, 이미지는 ros_gz_image	+3h (P1)
가우시안 플룸 확산 노드	필요	유지 — 시뮬레이터 무관 자체 노드	10h
배터리 소모 모델	필요	유지 (Return to Home에 포함)	—

순증감: P3에서 약 6h 절감, P1에서 약 5h 증가 → 거의 상쇄. 절감분은 stage1 이중화 대비 예비 시간으로 P2에 넘긴다.

부수 효과: 열화상을 합성이 아닌 실제 센서로 별도 링크에 장착하므로 RGB와의 위치 오차가 물리적으로 발생한다. M2-2(“위치 오차를 어떻게 수학적으로 보정했는가”)에 실측 Extrinsic 캘리브레이션으로 답할 수 있다.

③ 학습 주제 도출 — 우리가 모르는 것은 무엇인가?

우선순위: **P0**(없으면 프로젝트가 멈춤) / **P1**(품질·정량목표 좌우) / **P2**(보너스·심화)

#	Learning Issue	트랙	Phase	우선
LI-01	Docker on Apple Silicon: linux/arm64 ROS 2 Jazzy 이미지 빌드, GUI 우회(Foxglove/noVNC)	환경	P0	P0
LI-02	ROS 2 아키텍처: DDS, QoS(Reliable/BestEffort), Topic·Service·Action 선택 기준	시스템	P0/P5	P0
LI-03	ROS 배포판 간 이식성: Humble→Jazzy API 변경점, 오픈소스 패키지 포팅 판정 절차	환경/제어	P0/P2	P0
LI-04	ros_gz_bridge / ros_gz_image: gz-transport ↔ ROS 2 토픽 매핑, 메시지 대응표, 브리지 오버헤드	시스템	P1	P0
LI-05	Gazebo Harmonic SDF 월드 작성, Mesh(DAE/STL) 파이프라인, 제품 에셋 변환(Blender)	환경	P1	P0
LI-06	URDF/Xacro와 gz_ros2_control 연동, gz_frame_id 등 센서 부착 규약	환경	P1	P0
LI-07	Harmonic thermal 센서: min_temp/max_temp/resolution, heat signature로 발열 오브젝트 정의	환경/비전	P1/P3	P0
LI-08	PointCloud 필터링: 이상치 제거, 복셀 다운샘플링, 시간축 누적	제어	P2	P0
LI-09	Elevation Map 생성(grid_map / elevation_mapping CPU)과 발 디딤점 산출	제어	P2	P0
LI-10	4족 보행 기구학·동역학, Gait(Trot/Crawl/Pace) 안정성 차이	제어	P2	P0

#	Learning Issue	트랙	Phase	우선
LI-11	기성 4족 스택(CHAMP) 구조 분석 — 튜닝 가능 지점 vs 교체 가능 지점	제어	P2	P0
LI-12	자체 게이트 생성기: 발끝 궤적 생성 + 다리 역기구학 (stage1-B)	제어	P2	P0
LI-13	MPC: SRBD 모델링, QP 정식화, OSQP, Prediction Horizon 튜닝	제어	P2	P1
LI-14	Fall Detection(IMU 자세각·접촉)과 기립 관절 시퀀스	제어	P2	P1
LI-15	Nav2(Jazzy): costmap, planner/controller 플러그인, Behavior Tree	시스템	P4	P0
LI-16	야샷로그 게이지 인식: Hough 바늘 각도 vs 경량 CNN Keypoint	비전	P3	P0
LI-17	저조도·모션블러 전처리 (CLAHE, deblurring, 프레임 스테킹)	비전	P3	P1
LI-18	카메라 캘리브레이션과 열화상-RGB 정합 (Intrinsic/Extrinsic, Homography)	비전	P3	P0
LI-19	Viewpoint 생성 및 순찰 경로 최적화 (인식 비용함수, TSP, A*)	비전	P3	P1
LI-20	가스 확산: Gaussian Plume, 바람 벡터·난류 파라미터	시스템	P4	P0
LI-21	Source Seeking과 Local Optima 회피 (경사 상승, 나선 탐색, Infotaxis)	시스템	P4	P0
LI-22	상태 머신 설계와 배터리 기반 Return to Home 트리거	시스템	P4	P1
LI-23	TF2, use_sim_time, 다중 센서 시간 동기화(message_filters)	시스템	P5	P0
LI-24	rosbridge_suite + WebSocket 웹 대시보드, 이미지 압축 전송	시스템	P5	P1
LI-25	Sim2Real / Sim2Sim Gap: 접촉 파라미터 차이, Domain Randomization	제어	P6/P7	P1
LI-26	강화학습 보행: PPO/SAC, Isaac Lab 또는 Genesis	제어	P6	P2
LI-27	학습 정책 ONNX 내보내기 및 Gazebo 런타임 추론 이식	제어	P6	P2
LI-28	다중 로봇 네임스페이스 분리 및 구역 분할 협업 순찰	시스템	P6	P2
LI-29	EC2 GPU 스팟 운용, 비용 통제, 원격 배포	환경	P6	P2

우선순위 판단: ① 다른 작업의 선행조건인가 ② 정량 목표에 직결되는가 ③ 실패 시 대체 수단이 있는가. → **P0 15개를 Phase 0~1에 집중 학습**, P1은 해당 Phase 착수 직전, P2는 Phase 6 직전.

v2 대비 신설: LI-03, LI-04, LI-07, LI-12. 삭제: “Gazebo 커스텀 센서 플러그인 작성법”.

④ 자기주도 학습 — 필요한 지식을 스스로 학습하라

이름	담당
도훈	locomotion control
운학	inspection AI
태우	순찰 경로: critical path
수현	gazebo(virtual plant) + 관제(로봇 위치 3D맵, 로봇 시야 공유 라이브러리 탐색 등)
채현	gas safety

4-1. 트랙별 Learning Issue 배분

트랙	담당 LI	담당 산출물
환경	LI-01, LI-03, LI-05, LI-06, LI-07	docker/, simulation/worlds, simulation/models
제어	LI-08~LI-14	module1_locomotion/ 전체
비전	LI-16, LI-17, LI-18, LI-19	module2_inspection/ 전체
시스템	LI-02, LI-04, LI-15, LI-20~LI-24	module3_gas_safety/, dashboard/, 통신 규격
공통	LI-25	REPORT.md의 Sim2Real 장

태우의 순찰 경로(LI-19, viewpoint/TSP)는 비전 트랙과 맞닿는다. 운학과 인터페이스(/patrol_mission Action, 비용함수 입력)를 P3 착수 전에 맞춘다. 수현은 환경 트랙 (LI-05~07) + 관제(LI-24).

4-2. 조사 자료 출처

- 공식 문서: ROS 2 Jazzy, Gazebo Harmonic, gz-sensors(ThermalCameraSensor), Nav2, OpenCV, PyTorch, rosbridge_suite
- 브리지: gazebo-sim/ros_gz 메시지 타입 대응표
- 오픈소스: chvmp/champ 및 Jazzy 포크, ANYbotics/grid_map, elevation_mapping, gz_ros2_control, Unitree Go2 description
- 논문: MIT Cheetah 3 Convex MPC, Learning to Walk in Minutes, Infotaxis, Gaussian Plume
- 도구: Foxglove Studio, Isaac Lab / Genesis(보너스)

4-3. 학습 정리 템플릿

각 LI는 docs/learning/LI-XX.md에 축적한다.

```
# LI-13. MPC 기반 보행 제어

## 1. 핵심 개념 요약 (5줄 이내)
## 2. 우리 코드의 어디에 들어가는가
- 파일 경로 / 입력 토픽 / 출력 토픽
## 3. 선택한 방법과 근거 (대안 대비 비교)
## 4. 예상 어려움 / 미해결 질문
## 5. 출처 (URL·논문 제목·접근일)
## 6. 이 LI가 답하는 동료평가 질문 번호
```

- 개념 요약보다 “우리 코드의 어느 파일에 어떻게 들어가는가”를 반드시 한 줄 이상 쓴다.
- 6번(대응 평가 질문)을 채운다. 이 문서가 심층 인터뷰 답변 초안이 된다.

⑤ 실행계획 — 어떤 순서로 진행할 것인가?

5-1. Phase 로드맵 (필수 205h + 보너스 60h)

Phase	시간	주요 Task	완료 기준	평가 질문
P0. 개발환경	15h	arm64 네이티브 Ubuntu 24.04 + ROS 2 Jazzy + Gazebo Harmonic + Nav2 Docker / Foxglove-noVNC / 에셋 포맷 검증 / CHAMP Jazzy 1차 조사	docker compose up으로 빈 월드 + 뷰어, RTF 0.8 이상	—
P1. Virtual Plant	25h	복도·공장 월드 / 험지 장애물 / Go2 URDF + 센서(gpu_lidar·rgbd_camera·thermal-IMU) / ros_gz_bridge / 발열 설비 온도 태깅	모델링 최소 조건, ROS 2에서 전 센서 토픽 수신	DT-1
P2. Module 1	55h	Terrain Mapping 12h / stage1 15h / stage2 Python MPC 12h / stage3 C++ 10h(타입박스) / Fall Recovery 6h	험지·계단 통과 성공률, 넘어짐 시 자동 기립	M1-1~3
P3. Module 2	29h	thermal 설정 2h + 저조도/블러 3h / 게이지 15h / 열화상-RGB 정합 7h / Viewpoint 5h ※ 소개 중 3h는 P1로 이동	이동 중 판독 오차 ≤5%, 과열 배관 검출 데모	M2-1~3
P4. Module 3	30h	가우시안 플룸 + 농도장 10h / Source Seeking 12h / Return to Home 8h	임의 누출원 추적, 비상 트리거 시 복귀	M3-1, M3-2
P5. 통합 + 관제	30h	통신 규격 문서화 / rosbridge + 웹 대시보드 / 3모듈 통합 launch	스크립트 1개로 전체 시나리오, 대시보드 실시간 반영	통합-1, 통합-2, DT-2
P6. 보너스 ②③④	60h	③ RL 보행 30h(EC2) / ② 다중 로봇 15h / ④ 클라우드 관제 15h	각 보너스 데모 및 비교 결과	DT-3
P7. 문서화	15h	REPORT.md / README.md / 평가질문 답변 초안	제출 요건 7종 전부 충족	전체-1, 전체-2
예비	6h	stage1 이중화 대비 버퍼 (P3 절감분 이관)	—	—

5-2. 보너스 과제 범위 결정

보너스	결정	근거
① 실시간 대시보드	필수에 흡수 (P5)	Monitoring System이 이미 필수. 별도 시간 없음
③ RL 기반 보행	수행 (30h, EC2 스팟)	Sim2Sim 이식이 Sim2Real 질문의 실전 답변
② 다중 로봇 협업	수행 (15h)	네임스페이스 분리만으로 구현 가능
④ 클라우드 관제	수행 (15h)	① 대시보드를 EC2에 배포만 하면 한계비용이 낮음

비용 통제: EC2는 기동/중지 스크립트로만 조작, 스팟 인스턴스, \$30 예산 알람.

5-3. 평가 지표 (P7에서 정량 측정)

모듈	지표	목표
환경	빈 월드 / 통합 시나리오 RTF (arm64 네이티브)	0.8 이상 / 0.6 이상
Module 1	힘지 구간 넘어짐 횟수 / 10회 주행	1회 이하
Module 1	Fall Recovery 성공률	90% 이상
Module 1	Prediction Horizon별 자세 오차 RMS	stage2 실험 그래프화
Module 2	게이지 판독 오차율	5% 이내
Module 2	열화상-RGB 정합 픽셀 오차	10px 이내
Module 3	누출원 탐지 성공률 / 거리 오차	80% 이상 / 1m 이내
Module 3	Return to Home 성공률 (배터리 20% 트리거)	100%
통합	대시보드 End-to-End 지연	500ms 이하

5-4. 리스크 관리

순위	리스크	영향	대응
1	CHAMP 등 4족 오픈소스의 Jazzy 미지원	stage1 계획 변경	포크 검증 → 실패 시 1-B. 궤적 코드를 stage2와 공유. P2 착수 전 판정 필수
2	Apple Silicon Gazebo 성능 저하	RTF 하락	물리 스텝 조정, headless + Foxglove. 최악 시 EC2 CPU
3	ros_gz_bridge 오버헤드로 이미지 지연	P3·P5 실시간성	ros_gz_image 압축, 필요 토픽만 브리징
4	stage3 C++ MPC 미완성	없음 (stage1이 요건 충족)	타임박스 10h, 보고서에 시도·한계
5	제공 에셋 포맷 비호환	P1 지연	Blender 변환 우선 검증, 불가 시 자체 모델링
6	RL 정책의 Gazebo 이식 실패	보너스③ 품질	도메인 랜덤화, 실패 시 학습 곡선만 보고
7	EC2 비용 초과	예산	스팟 + 자동 중지 + 예산 알람
8	환경 재전환 유혹	대규모 재작업	Phase 2 완료 이후 Isaac Sim 전환 금지

5-5. 협업 규칙

- Git Flow: main / develop / feature/module{n}-{task}
- 모든 Task는 GitHub Issue 등록, Kanban 보드 관리
- 주 1회 통합 빌드 + 담당 LI 세미나(각 10분)
- Phase 완료 기준 통과 전에는 다음 Phase 착수하지 않음
- 의사결정 기록: Phase 0의 Jazzy 전환처럼 계획을 바꾼 판단은 docs/decisions/에 실측 근거와 함께 남긴다

부록 A. ROS 2 통신 구조 설계 초안

요구사항 #9 “통합” 두 질문과 DT-2에 대한 직접 답변. P3 착수 시점에 규격 확정.

A-0. Harmonic 브리지 계층 (v3 신설)

Gazebo Harmonic 센서는 gz-transport 토픽으로 발행되므로 ROS 2 노드가 직접 구독할 수 없다.

[Gazebo Harmonic] gz-transport

├─ ros_gz_bridge (LiDAR, IMU, JointState, Clock, cmd_vel)

└─ ros_gz_image (RGB, Depth, Thermal – 압축 지원)

▼

[ROS 2 Jazzy] 표준 sensor_msgs → 우리 노드들

필요한 토픽만 선별 브리징한다. /clock은 gz→ROS 단방향, 전 노드는 use_sim_time: true.

A-1. 토픽 설계

토픽	메시지 타입	발행 주체	주기	QoS
/clock	Clock	ros_gz_bridge	시물 스텝	Reliable

토픽	메시지 타입	발행 주체	주기	QoS
/scan/points	PointCloud2	gpu_lidar → bridge	10Hz	BestEffort, depth 1
/camera/rgb/image_raw	Image	rgbd → ros_gz_image	15Hz	BestEffort
/camera/thermal/image_raw	Image (L16)	thermal → ros_gz_image	9Hz	BestEffort
/imu/data	Imu	IMU → bridge	200Hz	BestEffort
/joint_states	JointState	gz_ros2_control	200Hz	BestEffort
/elevation_map	GridMap	terrain_mapping	5Hz	Reliable, depth 1
/cmd_vel	Twist	Nav2 / patrol_planner	20Hz	Reliable
/gait_mode	String	gait_switcher	이벤트	Reliable, TransientLocal
/inspection/gauge_reading	GaugeReading.msg	gauge_reader	이벤트	Reliable, TransientLocal
/inspection/thermal_alert	ThermalAlert.msg	thermal_fusion	이벤트	Reliable, TransientLocal
/gas/concentration	GasReading.msg	gas_simulation	5Hz	BestEffort
/gas/plume_map	MarkerArray	gas_simulation	1Hz	Reliable
/robot/battery	BatteryState	battery_model	1Hz	Reliable
/robot/status	RobotStatus.msg	mission_manager	2Hz	Reliable, TransientLocal

```
# GaugeReading.msg
std_msgs/Header header
string gauge_id
float32 value
string unit
float32 confidence
geometry_msgs/PoseStamped robot_pose
sensor_msgs/CompressedImage snapshot

# ThermalAlert.msg
std_msgs/Header header
string equipment_id
float32 max_temp_kelvin # thermal raw / resolution 환산
float32 threshold_kelvin
bool is_overheated
geometry_msgs/Point hotspot_position
```

thermal 센서는 온도(K)를 resolution으로 나눈 정수값을 이미지로 보낸다. thermal_fusion에서 raw × resolution = 켈빈 역산이 필요하며, 이 변환식을 REPORT 에 명시하면 M2-2 답변 정확도가 올라간다.

A-2. Service / Action 배분

방식	사용 대상	선택 이유
Topic	센서 스트림, 상태 브로드캐스트, 점검 결과	다수 구독자, 연속·비동기
Service	/set_gait_mode, /trigger_gauge_read, /reset_gas_source	즉시 응답, 짧은 요청-응답
Action	/patrol_mission, /seek_gas_source, /return_to_home, /fall_recovery	수 초~수 분, 진행률·중도 취소

A-3. 병목·충돌 방지 전략

1. 브리지 선별 — 필요 토픽만. 이미지는 ros_gz_image 압축.
2. 대역폭 분리 — 원본 이미지는 로컬만, 대시보드는 CompressedImage + 결과 메시지. rosbridge throttle.
3. QoS 분리 — 센서 BestEffort, 점검 결과·상태는 Reliable + TransientLocal.
4. 실행기 분리 — 제어 루프(200Hz)와 비전 추론을 별도 프로세스. 비전은 MultiThreadedExecutor + Reentrant.
5. 네임스페이스 — 보너스② 대비 처음부터 /robot1/....

A-4. 시간 동기화

- 전 노드 use_sim_time: true — Gazebo /clock을 단일 시간원. 브리지를 거치므로 Classic보다 더 중요하다.
- TF2: map → odom → base_link → {lidar_link, rgb_link, thermal_link, gas_sensor_link}

- RGB + 열화상: 15Hz vs 9Hz → ApproximateTimeSynchronizer 후 Extrinsic
- 대시보드는 수신 시각이 아니라 **stamp 기준** 정렬·표시

부록 B. Sim2Real / Sim2Sim 서술

실제 하드웨어 없이, 물리 엔진이 다른 두 시뮬레이터 간 정책 이식(Sim2Sim)을 수행한다. 보너스③에서 Isaac Lab/Genesis(PhysX)로 학습한 PPO를 ONNX로 보내 Gazebo Harmonic에서 실행한다.

차이 요소	문제	대응
접촉 마찰·강성	엔진마다 미끄러짐이 다름	마찰계수 0.4~1.2 랜덤화
링크 질량·관성	모델링 오차	질량 ±15% 랜덤화
액추에이터 지연·토크 한계	모터는 즉시 응답하지 않음	0~20ms 지연, 토크 포화
센서 노이즈·지연	시물 관측이 너무 깨끗함	IMU/관절각 가우시안 노이즈
관측 정규화 스케일	엔진별 단위·좌표 차이	ONNX 래퍼에서 입출력 규약 고정
브리지 지연	ROS 경로 추론 루프의 추가 지연	추론을 전용 노드/플러그인으로 흡 수 최소화

실제 로봇 이관 시 추가 요소: 레그 오도메트리 + IMU EKF, 실시간 커널, E-stop, 센서 캘리브레이션, 설비 도면 기반 3D 스캔. 열화상은 시물의 heat signature가 지정 온도를 그대로 렌더링하지만, 실제로는 방사율·반사·대기 감쇠가 있으므로 방사율 보정이 추가로 필요하다.

부록 C. 동료 평가 질문 ↔ 산출물 매핑

번호	질문 요지	답변 근거 산출물
M1-1	Elevation Map 노이즈 제거 및 발 디딤점	terrain_mapping/ + 필터 전후 비교 이미지
M1-2	MPC의 외란 감지와 토크 계산	stage2_simple_mpc_py/, Horizon별 RMS 그래프
M1-3	넘어짐 판정과 기립 시퀀스	fall_recovery/, IMU 임계 + 기립 성공률
M2-1	흔들림·조명 변화 전처리	gauge_reader/preprocessing, 전처리 유무 오차표
M2-2	열화상-실화상 위치 오차의 수학적 보정	thermal_fusion/, 실측 Extrinsic + 픽셀 오차 + 켈빈 역산식
M2-3	Viewpoint 가중치·비용 계산	patrol_planner/cost_function.py
M3-1	농도 구배 분석과 방향 벡터	source_seeking/, 탐색 궤적 + 국소최적 탈출 사례
M3-2	복귀 트리거와 경로 계획	return_to_home/, 상태 머신 + Nav2 설정
DT-1	DT 4구성요소와 실제 환경 반영	§1-4 매핑 표 + simulation/worlds
DT-2	센서 데이터 전달·시각화·동기화	부록 A (브리지, use_sim_time, TF2)
DT-3	Sim2Real 관점의 차이	부록 B + 보너스③ Sim2Sim
통합-1	모듈 간 병목·충돌 방지	부록 A-0 ~ A-3
통합-2	지연 최소화 및 실시간성	A-3 대역폭 분리 + E2E 지연 측정
전체-1	가장 어려웠던 기술적 문제	Apple Silicon 환경 + Jazzy/Harmonic 전환 (§1-5 실측)
전체-2	산업 현장 적용 시 고려사항	부록 B 하단 이관 요소

부록 D. 발표 운영안

슬라이드	내용	시간
1	표지 + 한 문장 정의 (§1-6)	0:30
2	① 문제·사용자·최종 결과물	1:30
3	① DT 4요소 매핑	1:00
4	① Phase 0 실측 → 스택 변경 — 차별화 1	2:00
5	② 의존 관계 그래프	1:30

슬라이드	내용	시간
6	② stage1 이중화 + 3단 리스크 분해 — 차별화 2	1:30
7	③ Learning Issue 29개 + 우선순위	1:30
8	④ 트랙별 배분 및 학습 템플릿	1:00
9	⑤ Phase 로드맵 + 평가 지표	2:00
10	⑤ 리스크 관리 + 보너스 범위	1:00
11	부록 A 통신 구조 1장 요약	1:00

발표 서사

“계획대로 실행했다”가 아니라 **계획 → 실측 → 근거 있는 변경**을 축으로 삼는다. 슬라이드 4~6이 이 서사의 두 축이다.

데모 영상: P2 종료 시 험지 통과, P3 종료 시 게이지 오버레이 + 열화상 퓨전, P4 종료 시 추적 경로, P5 종료 시 통합 + 대시보드. 각 Phase 완료 시점에 바로 녹화한다.